BRAINWARE UNIVERSITY

STUDENT RESULT AND CONTACT MANAGEMENT SYSTEM (WEEK 8)

1. Basic Information

Programs Covered:	Program 1: Student Result Management System Program 2: Mobile Contact and Call Management System
Name(s) of Student(s):	Soumyadwip Das (BWU/BTS/25/169) Ankita Paul (BWU/BTS/25/180) Sonia Naskar (BWU/BTS/25/195) Joyeta Mondal (BWU/BTS/25/214) Sayantan Bharati (BWU/BTS/25/503)
Programme & Semester:	B.Tech CSE & 3rd Semester
Course Name & Code:	Python Programming Lab (BES09013)
Name of the Guide/Supervisor:	MR. PRANAB GHARAI & MR. INDRANIL DAS
Project Date	04/09/2026

2. Introduction and Background

Week 8 covers two menu-driven, terminal-based record-management tools: a Student Result Management System for maintaining student records, subject marks, and grades, and a Mobile Contact and Call Management System for maintaining a contact book and a call log. Both programs keep core data-handling logic in dedicated functions separate from the input/output (menu) handling, and both use in-memory Python dictionaries as their data store.

3. Objectives of the Project (Week 8)

Program 1 - Student Result Management System

- Add and store student records (roll, name, section, course, year)
- Record subject-wise marks for each student
- Calculate percentage and grade from stored marks
- Display a full formatted result report for a student

Program 2 - Mobile Contact and Call Management System

- Add, view, search, update, and delete contacts
- Resolve a call target by contact name or phone number
- Log outgoing calls with a timestamp
- Display a formatted call history

4. Problem Statement / Driving Question

Program 1: How can student records, subject marks, and grades be tracked accurately for quick lookup and reporting?

Program 2: How can a contact book and call log be managed reliably, supporting lookups by either name or phone number?

5. Methodology (Week 8 Activities)

Program 1 - Student Result Management System

- Modelled each student as a dictionary entry keyed by roll number, holding a nested results dictionary
- Built core functions (add_student_data, add_marks_data, calculate_merit) with no I/O
- Built a get_grade / get_percentage pair to derive grade bands from average marks
- Built UI handler functions in a menu loop that call the core functions and print boxed results
- Tested add-student, add-marks, calculate-grade, and display-report flows

Program 2 - Mobile Contact and Call Management System

- Modelled contacts as a dictionary keyed by name, and call history as a list of records
- Built core functions (add_contact_data, search_contacts, update_contact_data, delete_contact_data)
- Built resolve_call_target to accept either a contact name or a raw phone number
- Built UI handlers for adding, viewing, searching, updating, deleting contacts, and making calls
- Tested add, search, update, call, and call-history flows

6. Expected Outcomes / Deliverables (Week 8)

- Working Student Result Management System with grade calculation
- Working Mobile Contact and Call Management System with call logging
- Clear separation of core logic and UI code in both programs
- Accurate percentage and grade computation from subject marks
- Reliable contact lookup by name or phone number

7. Core Logic

Program 1 - Student Result Management System

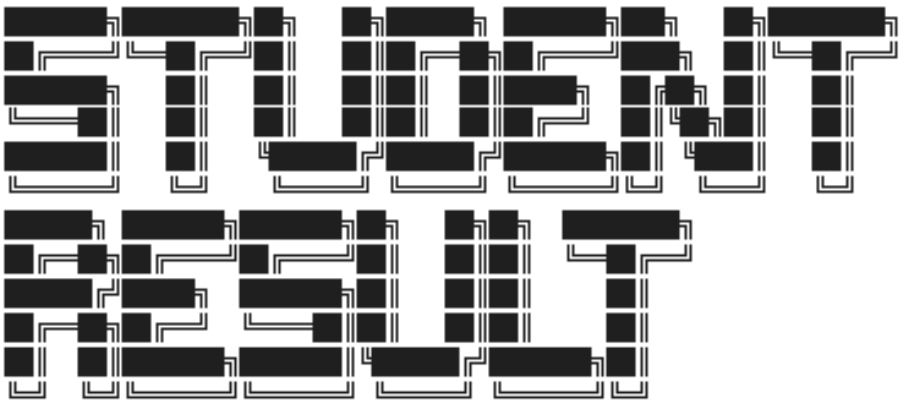
```
def add_student_data(roll, name, section, course, year):
    if roll in ROLLS:
        return False
    ROLLS.append(roll)
    all_students[roll] = {"name": name, "section": section,
                          "course": course, "year": year, "results": {}}
    return True

def add_marks_data(roll, subject, marks):
    if roll not in ROLLS: return False
    if not 0 <= marks <= 100: return False
    all_students[roll]["results"][subject] = marks
    return True

def get_grade(roll):
    subjects = all_students[roll]['results']
    if not subjects: return "N/A"
    percentage = sum(subjects.values()) / len(subjects)
    if percentage >= 90: return "A"
    elif percentage >= 75: return "B"
    elif percentage >= 50: return "C"
    else: return "F"

def calculate_merit(roll):
    if roll not in ROLLS or not all_students[roll]['results']:
        return None
    percentage = get_percentage(roll)
    grade = get_grade(roll)
    all_students[roll]["merit"] = {"percentage": percentage, "grade": grade}
    return percentage, grade
```

Program 2 - Mobile Contact and Call Management System

```
def add_contact_data(name, phone, email):
    if name in contacts:
        return False
    contacts[name] = {'phone': phone, 'email': email}
    return True

def search_contacts(term):
    term = term.lower()
    matches = {}
    for name, details in contacts.items():
        if term in name.lower() or term in details['phone']:
            matches[name] = details
    return matches

def resolve_call_target(target):
    if target.isdigit():
        for name, details in contacts.items():
            if details['phone'] == target:
                return name, target
        return target, target
    if target in contacts:
        return target, contacts[target]['phone']
    return None, None

def log_call(target, number):
    call_records.append({'type': 'outgoing', 'target': target,
                         'number': number, 'time': datetime.datetime.now()})
```

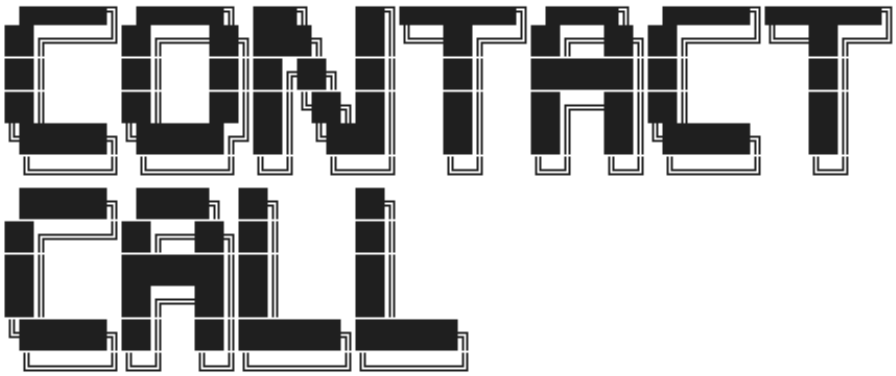
8. Output Screenshot

Program 1 - Student Result Management System



```
                    Terminal Student Result Management System

╔═══════════════════════════════════════════════════════════════════════════════╗
║   [1] Add New Student                                                         ║
║   [2] Add Subject Marks                                                       ║
║   [3] Calculate Grades                                                        ║
║   [4] Display Student Results                                                 ║
║   [5] Exit                                                                    ║
╚═══════════════════════════════════════════════════════════════════════════════╝
> Enter option: 1
> Enter student roll: 001
> Enter student name: ram
> Enter student section: b
> Enter student course: cse
> Enter student year: 2nd

╔═══════════════════════════════════════════════════════════════════════════════╗
║ STUDENT ADDED:                                                                ║
║                                                                               ║
║   Name : ram                                                                  ║
║   Roll : 001                                                                  ║
╚═══════════════════════════════════════════════════════════════════════════════╝

╔═══════════════════════════════════════════════════════════════════════════════╗
║   [1] Add New Student                                                         ║
║   [2] Add Subject Marks                                                       ║
║   [3] Calculate Grades                                                        ║
║   [4] Display Student Results                                                 ║
║   [5] Exit                                                                    ║
╚═══════════════════════════════════════════════════════════════════════════════╝
> Enter option:
```

Program 2 - Mobile Contact and Call Management System




```
                    Terminal Mobile Contact and Call Management System

  [1] Add Contact
  [2] View Contacts
  [3] Search Contact
  [4] Update Contact
  [5] Delete Contact
  [6] Make Call
  [7] View Call History
  [8] Exit

> Enter option: 1
> Enter contact name: ram
> Enter phone number: 9876543210
> Enter email (optional): ram@gmail.com

  CONTACT ADDED:

    Name  : ram
    Phone : 9876543210

  [1] Add Contact
  [2] View Contacts
  [3] Search Contact
  [4] Update Contact
  [5] Delete Contact
  [6] Make Call
  [7] View Call History
  [8] Exit

> Enter option: 8

  Goodbye!
```

9. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core language for both console systems
Python Dictionaries	In-memory storage for students, marks, contacts, and call records
Core/UI Separation	Business logic functions kept separate from input/print handlers
Helper Functions	Shared grade-calculation and call-target-resolution helpers
String Formatting (f-strings)	Aligned tabular display of contacts and call history

10. References

- Python Official Documentation: https://docs.python.org/3/
- Python Dictionaries Guide: https://docs.python.org/3/tutorial/datastructures.html
- Brainware University – Python Course Guidelines

11. Signatures

Signature of Student(s): **Date of Submission:**

Signature of Guide/Supervisor: